

PG Petrobras Initiation Tool (Simple) - User Guide

Command-line version of the PG Petrobras Initiation Tool. It automates the OrcaFlex static analysis of a **1st-end PLET initiation** onto a suction pile (SIP) receptacle: the PLET is lowered with a buoyancy module on a yoke, connected to the SIP with an initiation (bungee) wire, and landed in the receptacle while the vessel jacks the ramp down to the normal-lay angle.

The tool does three things:

1. **Build** an OrcaFlex base model from a project JSON file.
2. **Generate static steps**, either the full installation sequence (lay table) or a critical-step sensitivity study.
3. **Post-process** the step models into a lay table and engineering results workbook.

There is no user interface: all inputs live in two JSON files and each stage is one command.

1. How the tool works

The flowchart on the next page shows the complete flow. In short:

Stage	Command	What happens	Main output
Build	<code>run.bat build</code>	Reads and checks <code>input_data.json</code> , calculates PLET and buoyancy-module hydrodynamics, builds the OrcaFlex model on the selected vessel template	<code><Project>_<Pipe>_<Vessel>_Base_RevX.dat</code>
Full sequence	<code>run.bat full</code>	Optional landing check at the chosen ramp angle, then every installation step from vertical payout to the pipe on the seabed	<code>Statics_StepNNN_current-X.dat</code>
Critical step	<code>run.bat critical</code>	Solves the "PLET close to the receptacle" step for every combination of buoyancy, wire length, ramp angle, rollerbox opening and current, in parallel	<code>critical_*.dat</code>
Results	<code>run.bat results</code>	Re-runs statics on each step file and extracts lay table and code-check results, compared with the acceptance criteria	<code>Statics_Results_<current>.xlsx</code>
Snapshots	<code>run.bat snapshots</code>	Combines all step models into one model for visual review	<code>Steps/snapshots.sim</code>

Between **build** and **statics**, open the base model in OrcaFlex, check it and optimise the element (segment) lengths. The statics commands use the newest `*_Base_Rev*.dat` in the analysis folder unless `base_model` is set in `settings.json`.

PG Petrobras Initiation (Simple) - tool flowchart

run.bat <command> | inputs: input_data.json + settings.json | all outputs go to analysis_folder

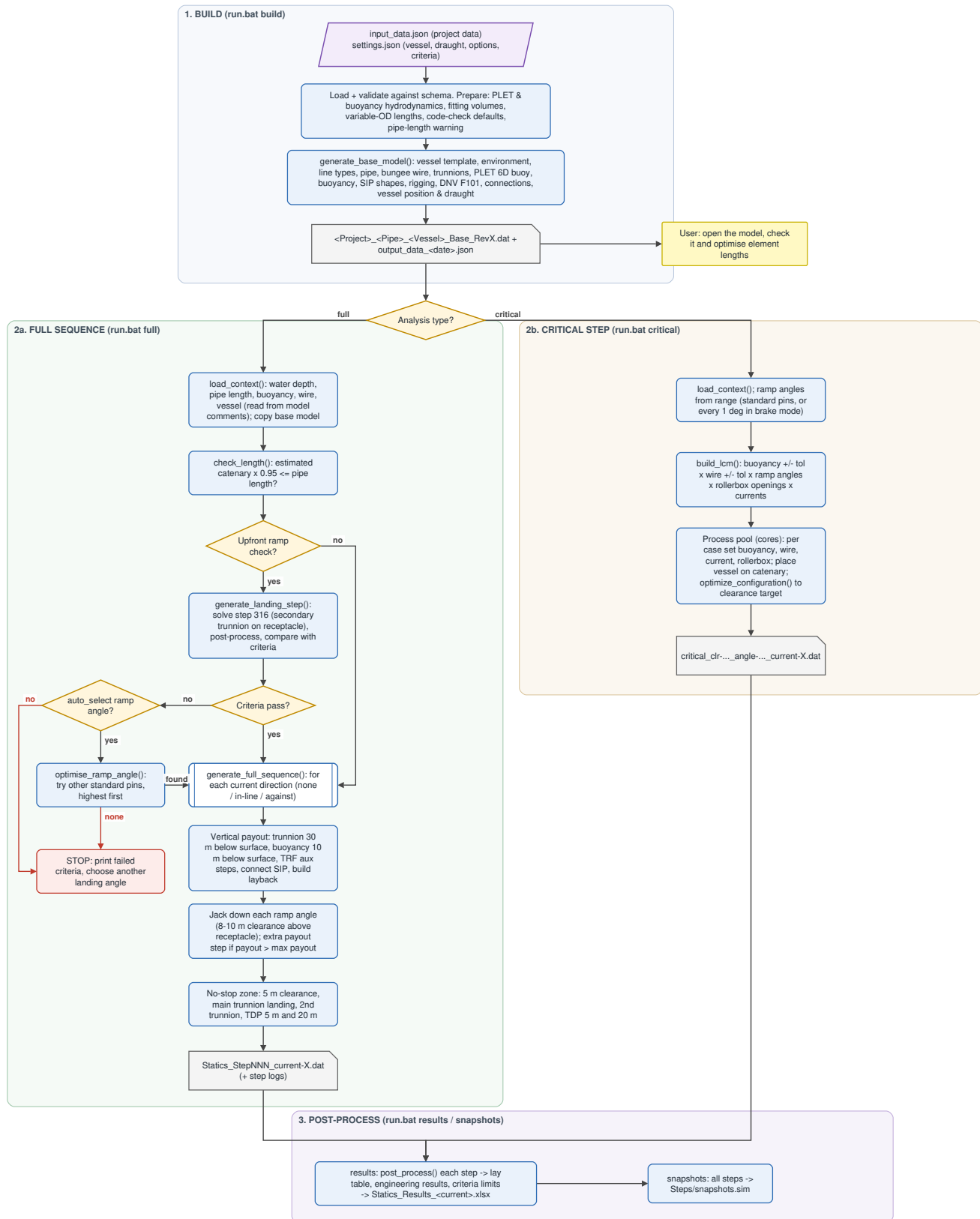


Figure 1 - Tool flowchart (also in docs/flowchart_overview.png / .svg)

2. Quick start

1. Install a **full, licensed OrcaFlex** on the PC. The OrcaFlex Demo cannot be driven from Python because it does not include `OrcFxAPI.dll`.
2. Copy or open the tool folder and run `run.bat info`. The first run creates a Python virtual environment (`venv`) and installs `requirements.txt`.
3. Put the project data in `input_data.json` (same format as the "Save Data" file of the old Streamlit tool, so old project files can be reused).
4. Edit `settings.json`: vessel, draught, analysis folder, statics options and criteria.
5. Run `run.bat build`, then open the base model in OrcaFlex and optimise the segment lengths.
6. Run `run.bat full` (lay table) or `run.bat critical` (sensitivity).
7. Run `run.bat results` (or `run.bat results --critical`) and review the Excel workbook.
8. Optionally run `run.bat snapshots` and open `Steps/snapshots.sim` to review all steps.

Every command prints its progress in the console. Optimisation logs (`stepNNN_optimization.log`) and error logs (`*error.log`) are written to the analysis folder.

3. Inputs

input_data.json (project data)

Section	Content
<code>general</code>	Client, project, pipeline, revision, job name, comments (written into the model comments)
<code>environment</code>	Water depth or seabed profile, seawater density, current profile, seabed stiffness
<code>lineTypes, variableOD</code>	Pipe line types (OD/WT/coatings, stress-strain: linear, Ramberg-Osgood or tabular) and variable-OD profiles
<code>lineData</code>	Pipe sections from the vessel to the PLET (line type, length)
<code>wireType, variableStiffness</code>	Initiation (bungee) wire, axial stiffness profile, rigging fittings and make-up
<code>structureData, sipData</code>	PLET weight, envelope, CoG, trunnions, perforation, connection points; SIP pile and receptacle geometry
<code>buoyancyData</code>	Yoke and buoyancy module (weights, dimensions, rigging length and stiffness)
<code>codeChecksData</code>	DNV-ST-F101 load factors, line-type factors, material and CRA properties

The PLET hydrodynamics, buoyancy-module volume and hydrodynamics, fitting volumes and line totals are recalculated from the geometry on every `build`.

settings.json (run options)

Key	Meaning
<code>vessel, draught</code>	7Vega, 7Oceans or 7Navica; RAO/draught name (<code>run.bat info</code> lists them)
<code>analysis_folder</code>	Where all outputs are written
<code>statics.contents_density, include_current</code>	Pipe contents (null = value in model); also run in-line and against-lay current
<code>statics.brake_mode</code>	7Vega only: 1 deg ramp increments / non-standard landing angle
<code>statics.use_rollerbox, rollerbox_openings_mm</code>	Add the vessel rollerbox model and its opening(s)
<code>full_sequence.*</code>	Landing ramp angle, maximum payout per jack-down, upfront ramp check, automatic ramp selection
<code>critical_step.*</code>	Ramp-angle range, clearance target, buoyancy and wire-length tolerances, number of cores
<code>criteria</code>	Acceptance limits (null = not checked): LCC at hang-off / sagbend / structure, strain, top tension, bollard pull, rollerbox load, layback, target-box utilisation, wire tension

4. Analysis logic

Full sequence (`run.bat full`)

Step	Configuration	How it is solved
Landing check (316)	Secondary trunnion in the receptacle at the selected ramp angle	<code>optimize_configuration</code> in contact mode, results checked against the criteria
3	Main trunnion 30 m below the surface, ramp vertical	Payout found with <code>fsolve</code> for the target depth
4	Buoyancy module 10 m below the surface (saved with and without buoyancy)	Payout for the target depth
Vega only	First jack-down to 90 deg with 3 m payout	Direct
Aux	Payout to install variable-OD sections (e.g. TRF)	Payout, vessel moved for zero hang-off moment when the PLET is near the seabed
5	Trunnion 10 m above the receptacle, SIP rigging connected	Payout for the target depth
6	Vessel moves to build layback	Vessel X for a 100 m minimum wire bend radius
7+	Jack down through each ramp angle keeping 8-10 m trunnion clearance	<code>optimize_configuration</code> ; an extra payout step is inserted when the payout exceeds the allowed maximum
No-stop zone	5 m clearance, main trunnion landing, 5 m pipe on seabed (or secondary trunnion landing), 20 m pipe on seabed	<code>optimize_configuration</code> in clearance, contact and touchdown modes

Each step is solved on a simplified pipe (4 sections) and then re-assigned to the real pipe make-up before it is stabilised and saved.

Critical step (`run.bat critical`)

Builds a load-case matrix from buoyancy uplift and wire length (with +/- tolerance), ramp angles in the chosen range, rollerbox openings and current directions. Each case is solved in its own process to the target PLET clearance and saved as `critical_...dat`. `run.bat lcm` writes the matrix to `lcm.csv` without running anything.

Solving one step

The loop below is used for every jack-down and landing step (Figure 2).

optimize_configuration() - solving one landing / jack-down step

Used by the full-sequence jack-down and landing steps and by every critical-step case

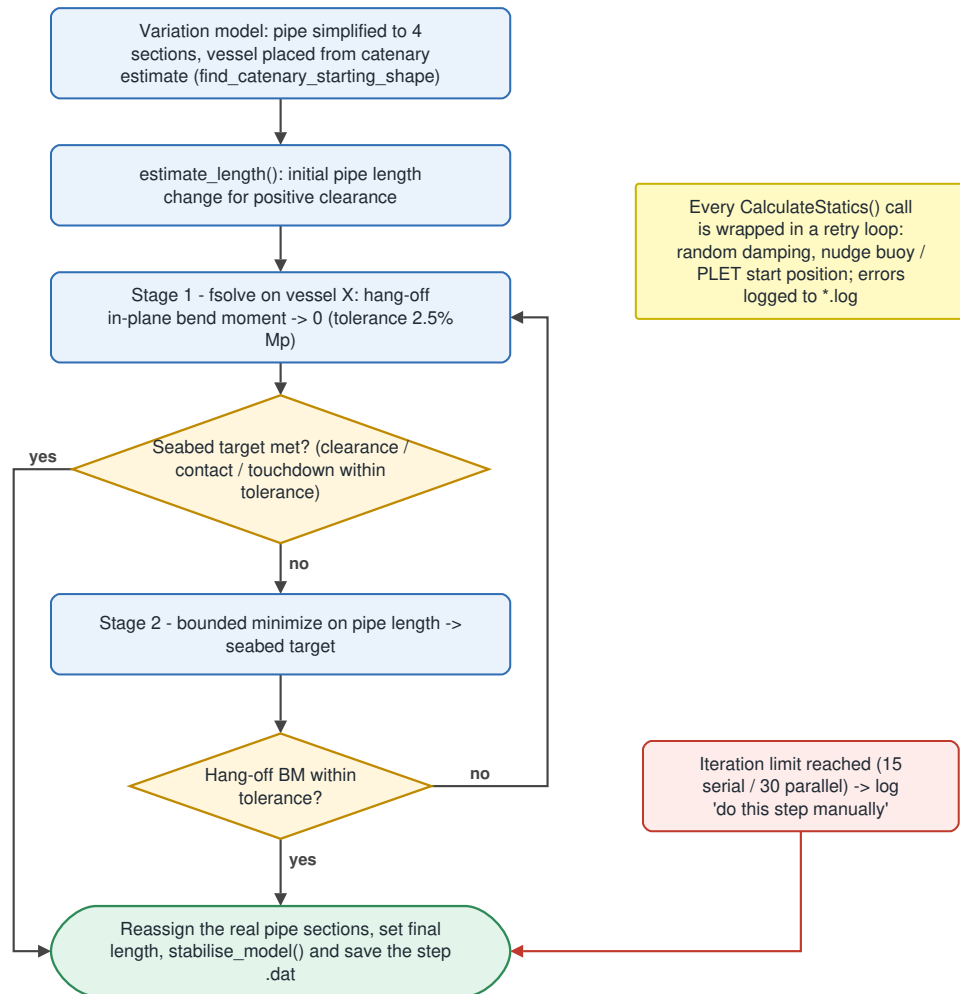


Figure 2 - optimize_configuration(): solving one step

5. Outputs

File	Content
<code>output_data_<date>.json</code>	Input data after the build calculations (traceability)
<code><Project>_<Pipe>_<Vessel>_Base_RevX.dat</code>	OrcaFlex base model
<code>Statics_StepNNN_current-<dir>.dat</code>	Full-sequence step models (<code>_NoBuoy</code> , <code>_aux</code> , <code>_unstable</code> variants)
<code>Statics_StepNNN_..._ramp-<angle>.dat</code>	Upfront landing check model
<code>critical_clr-..._current-<dir>.dat</code>	Critical-step case models
<code>stepNNN_optimization.log</code> , <code>*error.log</code> , <code>runtime.txt</code>	Solver history, convergence errors, run time
<code>Statics_Results_<dir>.xlsx</code>	Sheets: Job Data (or template header), Full Results, InitiationLayTable, Engineering Results
<code>Steps/snapshots.dat</code> , <code>.sim</code>	All steps in one model

6. Troubleshooting

Symptom	Cause / action
Exception: OrcaFlex not found	No full OrcaFlex install. Install it, or set <code>orcaflex_dll</code> to the full path of <code>OrcFxAPI.dll</code>
Required catenary length ... greater than the total pipe length	Add pipe length in <code>lineData</code> and rebuild
Step saved as <code>_unstable.dat</code>	Statics did not converge after 10 retries: open the file, adjust the start position and solve manually
"Iteration not progressing" in the console	The optimiser hit its iteration limit: check the step manually
Rollerbox error for 7Vega	Export the <code>_vega_RB_DB</code> Google Sheet to <code>assets/vessel_database/vega/_Vega_RB_DB.csv</code>
Results workbook without the company layout	<code>PetrobrasInitiation_Laytable_Template.xlsx</code> is not in <code>assets/</code> (Google Sheet in the original tool)

7. Code map

File	Responsibility
<code>run.py / run.bat</code>	Command line entry point, settings handling, command dispatch
<code>initiation/inputs.py</code>	Load, validate and prepare the input JSON
<code>initiation/hydro.py</code>	PLET and buoyancy-module hydrodynamics (DNVGL-RP-N103)
<code>initiation/vessels.py</code>	Vessel database: templates, hang-off points, draughts, pins, rollerbox coordinates
<code>initiation/base_model.py</code>	OrcaFlex base-model builder
<code>initiation/statics.py</code>	Full-sequence and critical-step static solvers
<code>initiation/postprocess.py</code>	Result extraction, criteria, Excel writer, snapshots
<code>initiation/catenary.py</code>	Catenary starting-shape estimate
<code>assets/</code>	Input schema and vessel database